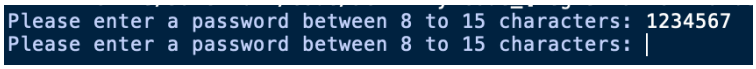
Task 1:

The name of my software is the password strength rating program. The purpose of the program is to check how strong a password is and give the user a rating, which can be very low, low, medium, high or very high. The program is written in Python. The intended users are people making a new password for their accounts that need to ensure their passwords are secure. It solves the issue of people using weak passwords such as all lowercase or uppercase which many may not know are not that secure. This program allows a user to get a rating to see whether they need a stronger password

Task 2:

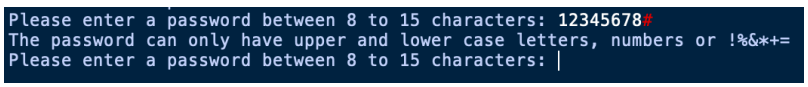
Requirement 1: The password must be between 8 – 15 characters

This solution meets the requirement, the length of the password is checked by using “len(password)”. If the password entered is not between 8 or 15 characters, it will print an error message and uses continue to go back to the beginning of the while True loop so the user can enter a new password. An example of this is when a 7 character password is tested, it was rejected and I had to enter a new one.



Requirement 2: The password can only contain upper and lower case letters, numbers or !%&*+=

This requirement has been met by checking each character with a for loop and checking it is a letter, by using (char.isalpha()), and checks for a number by using (char.isdigit()), or in the special string "!%&*+=". If any character is not allowed, valid is then set to False and an error message is shown so the user must try again. When I tested a password which contained an # in it, it was rejected.



Requirement 3: Add 10 point if the password is at least 10 characters long

This is done with if len(password) >= 10: total = total + 10. For example Password12 is 10 characters so it gets the extra 10 points.

Requirement 4: Add 10 points per special character

My program first checks each type of character first using the variables, upper_count, lower_count, number_count and special_count, it then calculates the total points from the counts. As an example Hello123! has 1 special, 3 numbers, 1 uppercase and 4 lowercase, so the total is 10 + 30 + 5 + 20 = 65, which is what my test plan expected.

```
Please enter a password between 8 to 15 characters: Hello123!
Your password rating is medium
```

Requirement 5: Remove 3 points per character if the password is entirely lowercase, entirely uppercase or entirely numbers.

This requirement is done by checking if the character count is equal to the length of the password, for example if lower_count has the same length as the password. When I tested "password" it got 40 points then lost 24 points, leaving 16, which ended up with a very low rating, which was expected.

Requirement 6: Output the correct rating (very low 20 or less, low 21 to 40, medium 41 to 70, high 71 to 80, very high over 80).

This is met by using an if, elif and else statement that checks the total from lowest to highest. Because it goes in order, it only needs to check the top number of each range. My tests showed very low, medium, high and very high ratings all working.

Task 3:

One strength of my program is the input validation. If the password that has been entered is not valid such as not having the correct characters in it, then it will not continue and will loop back. This is a strength because it means the program cannot give out any ratings for passwords that do not follow the rules, this stops wrong results. This benefits the user they are told what issues there are with their password in the error message, helping them fix it. This can be seen in my test plan where the short password and the # password tests were both rejected.

The accuracy is another strength of my program. The program checks each type of character first and then calculates the points, so the total is worked out the same way every time. For example, Hello123! was always coming out as 65 and medium both times. This means the user wouldn't need to worry that the rating system Is giving different answers for the same thing.

The program is also super user friendly. The user types in a password and gets a rating back in plain English, they don't need to understand anything about how the scoring works. As well as this the program is quite efficient because it only goes through the password a couple of times with simple for loops, so it gives the answer straight away.

Another strength is that useful variable names are used like upper_count and special_count, this makes the code easier to read and understand if someone else looks at it.

Task 4:

The first weakness is that the scoring system doesn't always give a realistic rating. For example, a password that is only numbers like 12345678 still gets a medium rating when its not a secure password. This may lead a user to use a weak password. In a future version I would change the points so that passwords with only one type of character can never get above low.

The second weakness is that the program doesn't allow for a wider range of special characters, for example, # or @. This could annoy the user because a good password could get rejected. It also doesn't check for repeated characters, so something like aaaaaaaa1! could still get a decent score. This could be changed by allowing more special characters and checking the password for more repeated characters.

The third weakness is that the program only gives the rating and doesn't show the total score or tell the user how to improve their password, while keeping it simple it may overall be easier for a user to see where their password could be improved. As well as this the program ends after one password has been entered, so if a user wants to try a new password they have to run the entire program again. This could be improved by printing the total and some tips and asking the user if they want to check another password at the end.

Task 5:

For testing I made a test plan before creating the program. I used normal data like Hello123!, boundary data like passwords that are exactly 8 and 15 characters and also 7 and 16 characters, and erroneous data like a password with a # in it. I also made tests to check the extra points for the length, the all lowercase, all uppercase and all numbers penalties, and a test for each rating.

While coding I found many bugs. The biggest issue was indentation. The valid check and break were inside the for loop, which meant it would stop after checking only the first character. My elif and else lines also weren't lined up with the if, so Python gave an indentation error. I also had break lines at the end of the program outside of any loop,

which gives an error. I fixed these by lining up the if, elif and else and making sure everything inside a loop was indented.

Overall testing improved the quality of my program a lot because it showed me that the program actually gives the right results and not just that it runs. Having the expected results worked out before coding meant I could easily see if something was wrong. Testing also showed me the weakness with 12345678 getting medium, which I wouldn't have noticed otherwise.

Task 6:

I think my code quality is good in some ways but could be better. I used variable names that are easy to understand such as password, total, upper_count, lower_count, number_count and special_count, this means it is easy to tell what each variable does.

The code is quite readable because it follows the same order as my flowchart and pseudocode, so it goes validation, then counting, then scoring, then the rating.

However, I didn't add a lot of comments to my code, this means someone else may find it overall harder to understand what each part does. In the future I would add comments.

Task 7:

The first improvement I would make is showing the user their total score and giving them help on how to improve their password, for example, "try adding a special character". This would improve the solution because the user would get a better understanding on why they got their rating. This would benefit the user because they could improve their password instead of guessing.

Another improvement would be letting the user check more than one password without restarting the program, by asking "Do you want to check another password?" at the end.

Task 8:

Overall, I think the program was successful. The program meets each of the requirements from the planning stage, it checks the password length and what characters are used, calculates the points and gives the rating. Testing was effective as I planned it before coding, it helped me find and fix a lot of bugs, mostly indentation ones.

The overall quality of the final product is good because it gives accurate results and is reliable, however the code could be improved with comments, and the scoring system has some weaknesses like all number passwords getting medium. I think the software is

suitable for its intended purpose of giving a user a quick idea of how strong their password is.